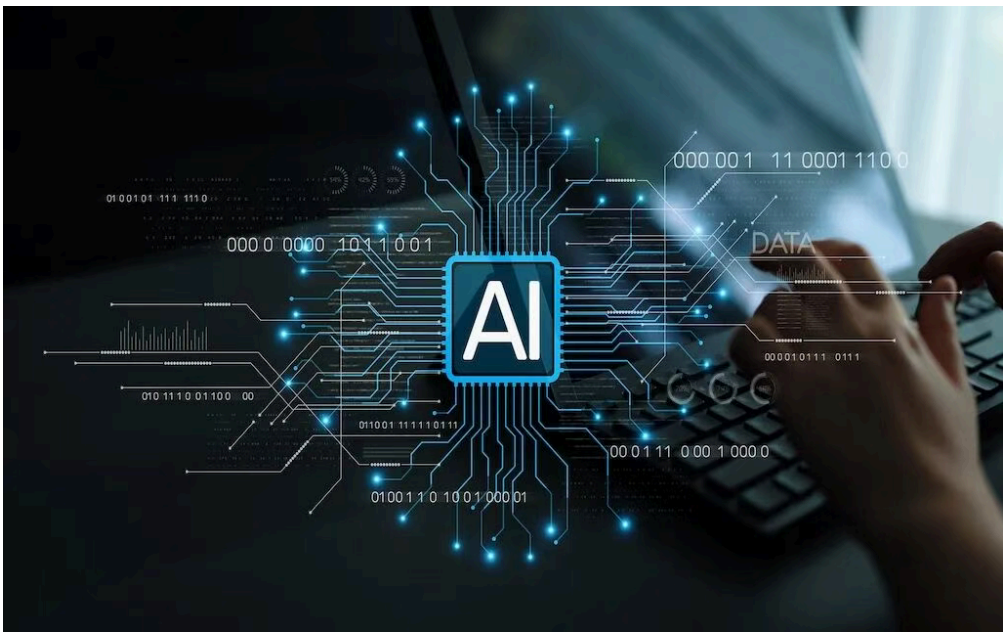
5 人 7 天干完 20 人数周的活：Spec-Driven Development 如何重新定义 AI 编程

王砚舒（彦纾） 阿里技术 2026年9月4日 03:08 浙江 37人



这是2026年的第 58 篇文章

（ 本文阅读时间：约 20 分钟 ）

前言：「5 人 7 天」 实验

本文分享一个案例。**5人、7天，用 Qoder 开发 QoderWork**，完成了传统情况下需要 20 人数周的工作量。它的时间线值得逐日复盘：

DAY 0：不写一行代码，只写 Spec。

团队花了一整天做四件事：定义 MVP 边界、拆解模块、为每个模块撰写 Spec、将所有 Spec 汇入 Repo Wiki。这一天的产出是零行代码，但它决定了后面六天的一切。

DAY 1-2：用 Qoder 完成架构开发。

框架与容器同步开发。每个人都是“Spec 工程师”，即定义代码应该长什么样的人；通过 Skill 驱动并行推进，Quest 模式同时执行多个开发任务。两天时间，系统骨架成型。

DAY 3-4：Spec 迭代增量需求。

发现新需求或 BUG？不用排期，立即写一个增量 Spec，委派 Quest 执行。AI 自动写代码并提交 PR，人负责 Review 与合并。单人日均处理多个 PR，效率抬升。

DAY 5-6：打磨、Dogfooding。

用 QoderWork 的早期版本测试 QoderWork 自身。发现问题就写 Spec，Quest 修复。这是一个自举式的正反馈循环。

DAY 7：正式发布上线。

七天，从零到一个可用的产品。

这个案例之所以震撼，不是因为“AI 写代码很快”，快只是表象。真正值得追问的是：为什么 5 个人能驾驭 AI 同时推进这么多并行任务而不失控?

答案藏在 DAY 0，那个看似 「什么都没做」的日子里，那一天写下的 Spec，就是整个项目的锚点。这套方法论有一个正式的名字：**Spec-Driven Development（SDD）**。

01
SDD 是什么：代码只是 Spec 的副产品

1.1 一句话定义

> Spec-Driven Development：将规格说明（Specification）作为唯一真实来源（Single Source of Truth），代码作为其派生产物。

用更直白的话说，就是先定义 WHAT，再让 AI 做 HOW。

如果我们做过 API-First Development、Design by Contract、或者严格的 TDD，就会觉得 SDD 似曾相识。但 SDD 的核心差异在于，它是为 **AI 编程时代** 量身设计的工程方法。

在传统开发中，Spec 写得好不好主要影响沟通效率；在 AI 编程时代，Spec 写得好不好 **直接决定代码质量**。因为 AI 不会追问”这个边界情况怎么处理“，它只会按照用户给的上下文尽力推断。推断对了是运气，推断错了是 Bug。

1.2 不是一个人的发明，而是一个时代的汇聚

SDD 没有单一发明者。2025 年，多个方向同时收敛到了这个理念：

- **Karpathy 的 Vibe Coding**（2025.2.2）作为反面参照，暴露了 "不管代码、只管 vibes" 的致命问题，倒逼社区思考 "AI 编程到底需要什么样的约束"；
- **GitHub 推出 Spec Kit**，提供了 agent-agnostic 的 SDD 工具链；
- **AWS 发布 Kiro**，成为第一个内置 SDD 工作流的 IDE；
- **Fission-AI 的 OpenSpec**，走轻量迭代路线；
- **阿里的 QoderWork**，通过 Qoder Quest 模式实践了 SDD 的规模化执行。

这种多方同时推动的局面，说明 SDD 是 AI 编程发展到当前阶段的**结构性需求**。

1.3 Microsoft 的那句点评

Microsoft 对 SDD 有一个精辟的评价，如下。

> "SDD is version control for your thinking."

传统的版本控制管理的是代码的演变历史，SDD 管理的是**思考的演变历史**：为什么做这个功能、边界在哪里、成功标准是什么。当代码可以被 AI 秒级重写时，真正有价值的是代码背后的决策。

02
SDD 完整流程拆解

2.1四阶段模型

SDD 的标准流程分为四个阶段：

```
Specify（规格定义）-> Plan（方案规划）-> Implement（代码实现）-> Validate（验证
```

阶段	主导者	核心产出	关键动作
Specify	人	spec.md	定义问题、边界、成功标准
Plan	人 + AI	plan.md	架构选型、模块划分、接口定义
Implement	AI	代码 + 测试	按 plan 逐任务实现
Validate	人 + AI	测试报告	自动化测试 + 人工 Review

人机分工的核心原则：人定义 WHAT，AI 实现 HOW。

在实践中，Specify 和 Plan 之间会有多轮迭代，Implement 和 Validate 之间也是持续循环的。

2.2 三文件体系：Spec Kit 的核心设计

GitHub 的 Spec Kit 提出了一个简洁的三文件体系：

spec.md —— 需求规格

这是「唯一真实来源」。它回答“做什么”和“为什么做”，不涉及“怎么做”。

```
# Feature: 用户权限管理模块
## Problem Statement
当前系统缺乏细粒度的权限控制。所有用户要么是管理员（全部权限），
要么是普通用户（只读权限）。产品团队需要支持至少 5 种角色，
以满足不同部门的差异化需求。
## Success Metrics
- 支持自定义角色，每个角色可配置不少于 20 种独立权限
- 权限校验 API 响应 P95 < 50ms
- 权限变更实时生效，无需用户重新登录
- 向后兼容：现有管理员/普通用户的权限行为不变
## User Stories
1. 作为系统管理员，我可以创建自定义角色并分配权限组合
2. 作为部门主管，我可以将部门成员批量分配到指定角色
3. 作为普通用户，我的权限变更后无需重新登录即可生效
## Acceptance Criteria
- [ ] RBAC 模型支持角色继承（最多 3 层）
- [ ] 单用户可拥有多个角色，权限取并集
- [ ] 提供权限变更审计日志，保留 90 天
- [ ] 权限校验支持通配符匹配（如 `resource:*:read`）
## Non-Goals
- 本期不实现跨组织的权限委托
- 不支持基于时间段的临时权限
- 不涉及 UI 层的权限管理界面（由前端团队单独出 Spec）
## Constraints
- 必须兼容现有的 OAuth2.0 认证流程
- 权限数据存储使用现有 PostgreSQL 实例，不引入新的存储组件
- 权限模型设计需参考 AWS IAM Policy 语法规范
```

我们需要注意这个 Spec 的几个特征，如下。

- 成功标准是可测试的："P95 < 50ms" 而非 "系统应该很快"；
- Non-Goals 明确划定了边界：告诉 AI "这些不要做"；
- Constraints 约束了技术选型：防止 AI 自作主张引入新组件。

plan.md —— 架构方案

基于 spec.md 生成的技术方案。这一步通常由 AI 起草，人负责审核修改。

```
# Plan: 用户权限管理模块
## Architecture Decision
采用 RBAC (Role-Based Access Control) 模型，使用 Casbin 作为权限引擎。
## Module Breakdown
1. `permission-model` - 权限模型定义与 Casbin 适配层
2. `permission-api` - RESTful API 层
```

```
3. `permission-cache` - Redis 缓存层（解决 P95 < 50ms 要求）
4. `permission-audit` - 审计日志模块
## Interface Contracts
### POST /api/v1/roles
### GET /api/v1/users/{userId}/permissions
### PUT /api/v1/roles/{roleId}/permissions
（接口详细定义省略）
## Risk Assessment
- 风险：角色继承层级过深可能导致权限计算性能下降
- 缓解：限制最大继承深度为 3 层，权限结果做预计算缓存
```

tasks.md —— 任务清单

将 plan 拆解为可执行的原子任务，每个任务对应一个可独立验证的交付物。

```
# Tasks
## Task 1: 数据库 Schema 设计
- 创建 roles、permissions、role_permissions、user_roles 四张表
- 支持角色继承的 parent_role_id 字段
- 验证：migration 脚本可在空库上成功执行
## Task 2: Casbin 适配层
- 实现 RBAC 模型的 Casbin 配置
- 支持通配符匹配
- 验证：单元测试覆盖率 > 90%
## Task 3: 权限校验 API
- 实现 GET /api/v1/users/{userId}/permissions
- 集成 Redis 缓存
- 验证：压测 P95 < 50ms
（后续任务省略）
```

2.3 constitution.md：不可变的项目原则

除了三文件体系，Spec Kit 还引入了一个重要概念 **constitution.md**。这是项目级别的“宪法”，定义了不可违背的约束条件，所有 Spec 都必须遵守。

```
# Project Constitution
## Immutable Principles
### 1. API Design
- 所有 API 遵循 RESTful 规范，版本化路径（/api/v1/...）
- 错误响应统一使用 RFC 7807 Problem Details 格式
- 所有 API 必须有 OpenAPI 3.0 文档
### 2. Security
- 所有用户输入必须经过校验和清洗
- 敏感数据（密码、Token）禁止出现在日志中
```

```
- 数据库查询必须使用参数化查询，禁止字符串拼接
### 3. Code Quality
- 单元测试覆盖率不低于 80%
- 所有公共方法必须有文档注释
- 禁止引入未经安全审计的第三方依赖
### 4. Infrastructure
- 所有服务必须支持优雅关闭（Graceful Shutdown）
- 配置项通过环境变量注入，禁止硬编码
- 日志格式统一使用结构化 JSON
```

constitution.md 的价值在于，它把团队的技术决策固化为 AI 的「潜意识」。没有它，每个 Spec 都需要重复声明“要用参数化查”“要写单元测试”这些基本约束。

03
Spec 怎么写：好 Spec 与坏 Spec 的生死线

Spec 写作是 SDD 中最关键、也最容易被低估的环节。经验印证了这一点——DAY 0 花整整一天写 Spec，正是因为 **Spec 的质量直接决定了 DAY 1-6 的效率。**

3.1 好 Spec 的六要素

要素	作用	示例
Problem Statement	定义 "为什么做"	"当前系统不支持细粒度权限控制"
Success Metrics	定义 "做到什么程度算完"	"P95 < 50ms，覆盖 20+ 权限类型"
User Stories	定义 "谁在什么场景下用"	"作为管理员，我可以创建自定义角色"
Acceptance Criteria	定义 "怎么验证"	"单用户多角色，权限取并集"
Non-Goals	定义 "什么不做"	"本期不做跨组织权限委托"
Constraints	定义 "技术约束"	"必须兼容现有 OAuth2.0 流程"

3.2 好 Spec vs 坏 Spec：一组对比

坏 Spec：

```
系统需要一个快速的搜索功能。搜索结果应该相关且准确。
界面要美观易用。
```

这个 Spec 有四个致命问题：

1. **模糊**——“快速”是多快？100ms？1s？10s？

2. **遗漏边界**——搜索什么？全文？标题？支持模糊匹配吗？
3. **缺乏理由**——为什么需要搜索功能？解决什么问题？
4. **混入了 HOW**——“界面要美观”是 UI Spec 的范畴，不应出现在后端功能 Spec 里。

好 Spec：

```
## Problem Statement
用户反馈在 10,000+ 文档的知识库中找到目标文档平均需要 3 分钟。
目标是将查找时间缩短到 10 秒以内。
## Success Metrics
- 搜索 API 响应时间 P95 < 200ms
- 搜索结果 Top-5 相关性准确率 > 85%（基于人工标注测试集）
- 支持中英文混合查询
## Non-Goals
- 不实现语义搜索（本期仅关键词匹配 + 分词）
- 不支持搜索附件内容（仅搜索文档标题和正文）
```

差异的本质：好 Spec 是可测试的，坏 Spec 是可解释的。

「系统应该很快」给了 AI 无限的解释空间，它可能选择一个“对它来说够快”的实现。「P95 < 200ms」则是一个硬约束，AI 必须确保实现满足这个指标，否则就是 fail。

3.3 粒度控制：一个实用的检验标准

Spec 的粒度很难拿捏。太粗，AI 会自作主张填补大量细节；太细，本质上就是在写伪代码，失去了 SDD 的意义。

Spec Kit 提出了一个优雅的检验标准：

> "用不同技术栈实现这个 Spec，Spec 是否仍然有效？"

如果 Spec 写了“使用 Redis 的 ZSET 结构存储排行榜”，那它只对 Redis 方案有效，换成 PostgreSQL 实现就失效了。这说明把 HOW 混进了 WHAT。

如果 Spec 写了“排行榜需支持实时更新，延迟不超过 1 秒，支持 Top-100 查询”，那无论底层用 Redis、PostgreSQL 还是自研存储，这个 Spec 都成立。这才是正确的粒度。

当然，Constraints 部分可以出现技术约束，但那是“外部限制”（比如“必须使用现有 PostgreSQL 实例”），不是「实现方案」。

3.4 淘特团队的实战经验

淘特团队在实践 SDD 时发现一个重要规律：**Spec 编写需要 3-5 次迭代才合格。**

第一版 Spec 通常问题百出，比如遗漏边界条件、成功标准模糊、Non-Goals 不够明确等。让 AI 基于第一版 Spec 生成 plan 之后再回头审视 Spec，往往能暴露出大量盲点。这个「Spec -> Plan -> Review Spec -> 修改 Spec -> 重新 Plan」的循环通常要跑 3-5 轮。

这是把传统开发中“开发到一半发现需求有问题”的代价前移到了成本最低的阶段，改一行 Spec 的成本远低于改一百行代码。

04
工具生态全景

SDD 已经形成了一个快速发展的工具生态，以下是主要工具的对比。

维度	Spec Kit (GitHub)	OpenSpec (Fission-AI)	Kiro (AWS)	QoderWork (阿里)
定位	Agent-agnostic 框架	轻量迭代工具	SDD-native IDE	Qoder Quest 执行引擎
Agent 支持	8+ Agent	25+ 工具	内置 Agent	Qoder 生态
核心特点	三文件体系 + constitution	轻量、快速迭代	完整 IDE 集成	Spec + Quest 并行执行
适用场景	通用项目	小型快速迭代	AWS 生态项目	阿里生态项目
学习曲线	中	低	中	中（需理解 Quest）
核心理念	Spec 即文档	Spec 即对话	Spec 即工作流	Spec 即任务单

4.1 各工具的差异化选择建议

追求通用性：选 Spec Kit。它不绑定任何特定 Agent，支持 Claude、GPT、Gemini 等主流模型，三文件体系清晰易懂。43.7k Star 说明社区认可度高。

追求轻量和速度：选 OpenSpec。它的核心理念是 "Spec 不需要一步到位，可以在对话中迭代完善"，对小型项目非常友好。

在 AWS 生态内：Kiro 是最省心的选择。SDD 工作流直接内嵌在 IDE 中，不需要自己搭建工具链。

QoderWork 的 Quest 模式是最贴合 SDD 的执行引擎，「5 人 7 天」案例就是明证。Spec 写好后直接委派 Quest 执行，AI 自动写代码、提 PR、跑测试。

05
实战数据：成功与失败都摆上台面

5.1 成功案例的硬数据

API 变更效率提升

一篇 arXiv 论文记录了金融领域的 SDD 实践：API 变更周期缩短**75%**。原因在于当 API 的 Spec 已经完整定义了接口契约、错误码、数据格式时，AI 可以直接生成符合 Spec 的代码，省去了大量的沟通和返工。

代码错误率下降

研究数据显示，人工精炼 Spec 可以将 LLM 代码错误减少 **50%**。这说明好的 Spec 消除了 AI 的“猜测空间”，错误往往来自 AI 对模糊需求的错误推断。

Stripe 的规模化实践

Stripe 通过 Harness Engineering 方法（SDD 是其重要组成部分）交付了 **1300 个 AI PR**。这个数字的意义在于「可控」，1300 个 PR 没有引发系统性问题，说明 Spec 约束下的 AI 编程是可以规模化的。

5.2 失败案例的警示数据

无 Spec 约束的安全灾难

Veracode 2025 年的报告揭示：**45% 的 AI 生成代码包含安全漏洞。**注意前提，这是无 Spec 约束时的数据。当 Spec 中明确定义了安全约束（如“必须使用参数化查询”“敏感数据禁止出现在日志中”），漏洞率会大幅下降。

这 45% 和前面的「错误减少 50%」形成了鲜明对比。

场景	关键指标
无 Spec 约束的 AI 编程	45% 代码含安全漏洞
有 Spec 约束的 AI 编程	代码错误减少 50%

代码重复率的隐性成本

GitClear 对 2.11 亿行代码的研究发现，AI 编程时代的代码重复率 **4 年增长 4 倍**。这不难理解，AI 倾向于「复制粘贴式」的解决方案，因为它的训练数据中充满了类似的模式。SDD 通过 constitution.md 中的代码规范约束和 plan.md 中的模块化设计，可以在一定程度上缓解这个问题，但无法完全消除。

06
SDD vs Vibe Coding：一场必须正面交锋的辩论

6.1 什么是 Vibe Coding

2025 年 2 月 2 日，Andrej Karpathy 在社交媒体上创造了 **Vibe Coding** 这个概念。

> "Forget that the code even exists."（忘掉代码的存在。）

Vibe Coding 的核心主张是，不要去读代码、不要去理解代码，用自然语言描述你要什么，让 AI 全权处理。如果报错了，就把错误信息扔给 AI，让它自己修。

这听起来很诱人。**在某些场景下，它确实有效**，比如快速原型验证、个人工具脚本、一次性的数据处理任务。

但问题在于，**它不可持续。**

6.2 "三个月墙"：Vibe Coding 的生死劫

社区观察到一个反复出现的模式，被称为**「三个月墙」**，如下。

阶段	时间	状态	典型表现
兴奋期	1–3 个月	高产出	"AI 太神了！一天搞定一个功能！"
平台期	4–9 个月	停滞	"为什么新功能总是破坏旧功能？"
衰退期	10–15 个月	崩溃	"这坨代码没人能维护了，不如重写"

为什么会撞墙？因为 Vibe Coding 本质上是 **零上下文的编程**。

当项目只有几百行代码时，AI 可以“看到”全貌，它的推断和猜测大概率是对的。但当项目膨胀到几万行、几十个模块、几百个接口时，AI 的上下文窗口装不下全部代码，它开始基于局部信息做决策，而这些决策往往和其他模块冲突。

SDD 的 Spec 恰好解决了这个问题：**Spec 是代码的压缩表示**。10 万行代码的项目，它的 Spec 体系可能只有几千行文本。AI 可以轻松读完所有 Spec，理解全局约束后再动手改代码。

6.3 核心差异对比

维度	Vibe Coding	SDD
核心假设	AI 能理解你的意图	AI 需要明确的规格才能正确执行

维度	Vibe Coding	SDD
启动速度	极快	较慢（需要先写 Spec）
可维护性	差（无文档、无约束）	好（Spec 即文档）
可协作性	差（只有作者知道 "vibes"）	好（Spec 是共享语言）
安全性	差（45% 安全漏洞）	较好（Spec 约束安全边界）
适用规模	小项目（< 1000 行）	中大型项目
天花板	三个月墙	取决于 Spec 体系质量

6.4 我的判断：不是非此即彼

Vibe Coding 和 SDD 是一个**光谱的两端**。对于一个验证想法的周末项目，写 Spec 是过度工程化；对于一个要长期维护的生产系统，Vibe Coding 是慢性自杀。

更务实的选择是**混合策略**：

1. **探索阶段**用 Vibe Coding——快速试错，验证可行性；
2. **一旦决定要做**，立刻补 Spec——把探索阶段的发现固化为规格；
3. **进入正式开发**后严格 SDD——所有变更先改 Spec 再改代码。

实践就是这个策略的一个变体。DAY 0 写 Spec 之前，团队必然已经对 QoderWork 的产品形态有了充分的探索和讨论（那属于更早的 Vibe 阶段），但一旦决定动手做，立刻切换到 Spec-Driven 模式。

07
SDD 与 Harness Engineering 的关系

AI 编程方法论的演进，存在一条清晰的三层递进链：

```
Prompt Engineering -> Context Engineering -> Harness Engineering
```

- **Prompt Engineering**：关注「怎么写好一条指令」；
- **Context Engineering**：关注「怎么给 AI 提供足够且精准的上下文」；
- **Harness Engineering**：关注「怎么构建一个系统性的框架来约束和驱动 AI」。

SDD 位于 **Context Engineering 和 Harness Engineering 的交叉地带。**

从 Context Engineering 的角度看，Spec 就是一种高度结构化的上下文提供方式，它不把所有代码丢给 AI（那太嘈杂），也不只说一句话（那太模糊），转而提供了一个恰到好处的信息

密度。

从 Harness Engineering 的角度看，SDD 是 HE 的一个重要应用模式：constitution.md 是约束层，spec.md 是需求层，plan.md 是执行层，tasks.md 是调度层。这四层组合在一起，构成了一个完整的「AI 驾驭框架」（Harness）。

Stripe 交付 1300 个 AI PR 的实践就是 HE 方法论的典型案例，其中，SDD 是“确保每个 PR 都有明确规格和验收标准”的关键组件。

08
五大陷阱与局限性：诚实面对 SDD 的阴暗面

在拥抱 SDD 之前，我们需要正视以下五大陷阱。

陷阱一：过度规格化（Over-Specification）

症状：Spec 写得比代码还长，每个实现细节都规定得死死的。

后果：本质上退化为“用自然语言写伪代码”，完全失去了 SDD 「人定义 WHAT、AI 实现 HOW」的核心优势。

缓解：用前文提到的粒度检验标准，“换一个技术栈实现，这个 Spec 是否仍然有效？”如果答案是否，说明在 Spec 里混入了 HOW。

陷阱二：规格腐烂（Spec Rot）

症状：代码已经迭代了十几个版本，但 Spec 还停留在 V1。

后果：Spec 和代码脱节，新人看 Spec 理解的和看代码理解的是两个系统。当需要 AI 做增量开发时，它基于过时的 Spec 生成的代码必然和现有代码冲突。

缓解：团队的做法值得借鉴。DAY 3-4 中，“发现需求或 BUG 立即写增量 Spec”。Spec 的更新是和代码变更同步的日常操作。

陷阱三：规格官僚化（Spec Bureaucracy）

症状：改一个按钮的颜色也要走「写 Spec -> Review Spec -> 更新 Plan -> 生成 Tasks」的全流程。

后果：团队开始绕过 Spec 流程直接改代码，SDD 名存实亡。

缓解：区分**重大变更**和**微小变更**，不是所有变更都需要完整的 Spec 流程。一个实用的判断标准是，如果这个变更可能影响其他模块的行为，就需要 Spec；如果它是纯粹的局部修改（改文

案、调样式），直接改就行。

陷阱四：虚假信心（False Confidence）

症状：因为有了详尽的 Spec，团队放松了对 AI 生成代码的审查。

后果：Spec 只定义了「做什么」，它不能保证 AI 的实现是正确的、安全的、高效的。45% 的安全漏洞数据提醒我们，即使有 Spec，AI 代码仍需要严格的 Review。

缓解：Spec 替代的是需求文档，不是 Code Review。 SDD 的 Validate 阶段是整个流程中最后也最重要的安全网。

陷阱五：工具复杂性（Tool Overhead）

症状：为了「做好 SDD」，团队引入了 Spec Kit + Kiro + CI/CD 集成 + 自定义校验脚本 + Spec Linter...

后果：工具链本身成为了负担，团队花在配置工具上的时间超过了写 Spec 的时间。

缓解：从最简单的方案开始。 一个 spec.md 文件 + 一个你已经在用的 AI Agent，这就够了；不要为了 SDD 而 SDD。

正面回应：「SDD 是 Markdown 版的瀑布模型」批评

这是 SDD 面临的最尖锐的批评，值得认真对待。

批评者的逻辑是，SDD 要求先写完 Spec 再写代码，这本质上就是瀑布模型。先完整定义需求，再线性地实现。而瀑布模型早已被敏捷运动证伪，SDD 不过是换了个马甲。

我的判断：这个批评有一半道理，但结论是错的。

「有一半道理」是因为，如果我们把 SDD 实践成“花两周写一个巨大的 Spec，然后完全不改地执行”，那确实是瀑布模型的翻版，必然失败。

「结论是错的」是因为，**SDD 的 Spec 是活的，不是死的。** 团队的 DAY 3-4 就是最好的反驳，Spec 是随时可以增量更新的活文档。发现新需求？加一个增量 Spec；发现 BUG？补一个约束条件。这更接近“持续需求细化”。

更本质的区别在于迭代粒度：

- 瀑布模型的迭代粒度是 **整个项目**（需求全部定义完 -> 全部开发完 -> 全部测试完）
- SDD 的迭代粒度是 **单个功能模块**（一个模块的 Spec -> 该模块的 Plan -> 该模块的实现 -> 该模块的验证，然后进入下一个模块）

SDD 实际上更接近**「Spec 级别的迭代」**，每个 Spec 就是一个最小可交付单元，写完就执行、执行完就验证、验证完就下一个。这和敏捷的 Sprint 理念是兼容的，只是把 User Story 升级成了更结构化的 Spec。

另外一个反对「瀑布论」的事实是：**SDD 中 Spec 的平均生命周期远短于瀑布模型中需求文档的生命周期。** 一个 Spec 从编写到被 AI 实现可能只需要几小时甚至几分钟，而瀑布模型的需求文档生命周期以月计。

当然，SDD 确实 **有瀑布化的风险**。但这是实践问题，不是方法论问题。一把菜刀可以切菜也可以伤人，问题不在刀上。

09
SDD 的未来：三级光谱模型

SDD 并不是一个固定不变的方法论，它正在沿着一个三级光谱演进：

级别	名称	当前状态	特征	代表实践
L1	**Spec-First**	当前主流	编码前写 Spec，但 Spec 和代码可能漂移	Spec Kit, 大多数团队
L2	**Spec-Anchored**	先进实践	Spec 和代码持续同步，测试强制执行一致性	Kiro 的内置工作流
L3	**Spec-as-Source**	未来愿景	人只编辑 Spec，代码完全由 AI 生成和维护	尚无成熟实践

L1 Spec-First：编码前写 Spec，但 Spec 可能漂移

这是目前大多数团队的实践状态。Spec 在项目初期发挥了巨大价值，但随着项目推进，Spec 和代码之间的漂移几乎不可避免，因为没有自动化机制来强制保持一致。

L2 Spec-Anchored：持续同步，测试强制执行

这是当前先进团队正在探索的状态。核心思路是，**用自动化测试来锚定 Spec 和代码的一致性。**

具体做法：从 Spec 的 Acceptance Criteria 自动生成测试用例，任何代码变更都必须通过这些测试。如果代码变了但测试（即 Spec 的自动化映射）没变，CI 会拒绝合并。这就像“类型系统”防止类型错误一样，“Spec 测试”防止 Spec 漂移。

AWS Kiro 正在朝这个方向做，它的 SDD 工作流中内置了 Spec -> Test 的自动生成链路。

L3 Spec-as-Source：人只编辑 Spec，代码完全生成

这是 SDD 的终极愿景，**代码是被「编译」的——从 Spec 编译而来。**

在这个范式下，人类工程师的工作完全聚焦在 Spec 层面。需要改功能？改 Spec；需要修 Bug？改 Spec；需要重构？改 Spec。AI 负责将 Spec 的变更自动反映到代码中。

这听起来像科幻，但想想看，团队的 DAY 3-4 已经有了 L3 的雏形：发现 BUG -> 写增量 Spec -> Quest 自动修复代码并提 PR。如果这个循环足够顺滑、覆盖足够全面，L3 就不再是幻想。

Thoughtworks 技术雷达也在关注这个趋势。在 Vol.33（2025.11）中，SDD 被放在 **Assess** 环，这意味着“值得研究，但尚未准备好全面采用”。到了 Vol.34，越来越多团队开始采用 SDD，它正在向 Trial 环迁移。

10
结语：DAY 0 是最贵的一天，也是最值的一天

回到前文所说的「5 人 7 天」案例。

如果只看表面，我们会觉得奇迹发生在 DAY 1-6：AI 飞速写代码、自动提 PR、并行推进多个任务。

但真正的关键手在 DAY 0。

DAY 0 的一天「什么都没做」，实际上做了最重要的一件事：把人类的思考，即需求边界、成功标准、技术约束、非目标清单结构化地固化到了 Spec 中。

这些 Spec 成为了后面六天的导航系统。5 个人之所以能驾驭 AI 并行推进大量任务而不失控，正是因为每个 Quest 都有一个清晰的 Spec 锚定，AI 知道要做什么、不做什么、做到什么程度算完。

这就是 SDD 的本质价值。**它不让 AI 变聪明，它让 AI 变可控。**

在 AI 编程能力以月为单位飞速进化的今天，「让 AI 变聪明」不是我们需要操心的事，模型厂商会“卷”。人们真正需要操心的是：**当 AI 越来越强大时，能不能驾驭它？**

SDD 给出的答案是，与其试图理解和控制 AI 的每一行代码，不如把精力放在定义那个「WHAT」——WHAT 永远是人类的领地。

从今天开始，试着在下一个功能需求前先写一个 spec.md。

不需要完美，3–5 次迭代后它会变好；不需要工具链，一个 Markdown 文件就够了；不需要全员推广，从一个人、一个模块开始就行。

DAY 0 是最「贵」的一天，也是最「值」的一天。



欢迎留言一起参与讨论~

阅读 5840

留言 8

写留言

西瓜吃了夏天 广东 9月4日 1

写完了，一堆bug，性能问题完全不提，就是说写完了，写完了，好用能不能用两回事

奥 浙江 9月13日

AI Coding 也就只能出 Demo，针对性能和体验，无法输入给 AI 充足的上下文。

D S 上海 9月21日

还是需要人知道怎么去实现的，ai写的代码cr后要改好多，同样一句话，它写出来的代码每次都不一样

海妖 上海 9月13日

spec 流程债务太重怎么解决呢

小李讲道理 山东 9月8日

@元宝 总结一下

元宝 广东 9月8日

文章讲的是 Spec-Driven Development（SDD，规格驱动开发），核心是用 Spec 来驾驭 AI 编程。帮你把要点拆一下：

1、什么是 SDD：把规格说明（Spec）作为唯一真实来源，人定义 WHAT，AI 实现 HOW。代码只是 Spec 的副产品。

2、案例引子：5 人 7 天做出 QoderWork，完成了传统 20 人数周的量。关键不在 AI 快，而在 DAY 0 花一整天只写 Spec，后面六天所有任务都有锚点。

3、标准流程：Specify（需求）→ Plan（方案）→ Implement（实现）→ Validate（验证）。GitHub Spec Kit 的三文件体系是 spec.md、plan.md、tasks.md，再加一个不可变的 constitution.md。

4、好 Spec 的生死线：可测试，而不是可解释。要写"P95 < 200ms"，别写"系统要快"。还要有 Non-Goals 和 Constraints，明确告诉 AI 什么不做、受什么限制。

5、SDD vs Vibe Coding：Vibe Coding 适合小项目快速验证，但会撞上"三个月墙"；SDD 适合中大型长期维护的项目。实务上是光谱两端，可以混合用。

6、五大陷阱：过度规格化、规格腐烂、规格官僚化、虚假信心、工具复杂化。

7、未来演进：L1 Spec-...

内容由AI生成

大石头 北京 9月4日

上面的套件用来做技术评审都不行，技术评审文档有合适的方案么

Teemo 浙江 9月4日

虽然但是 这篇文章 ai 味有点太浓了 不考虑 human-writing 一点嘛🤦